POC-07 - Azure Machine Learning

MLflow + LLMOps / MLOps

Beginner Reference Guide - End-to-End Implementation, Verification, Troubleshooting, Batch Deployment, and Cleanup

Synthetic shipment delay prediction with risk_score

Guide map

- 1. Objective, architecture, data contract and cost guardrails
- 2. Project structure and the role of each Python file
- 3. Poetry environment creation and complete local verification flow
- 4. Azure Portal: resource group and Azure Machine Learning workspace
- 5. .env, Azure CLI authentication, verify_config.py and connection checks
- 6. Register data, track two MLflow experiments, compare and register model
- 7. Local/Azure scoring and risk_score validation
- 8. Batch compute, endpoint, custom environment, deployment and invocation
- 9. Real errors encountered: auth mismatch, stale .env, batch child failure, image build failure
- 10. Final success evidence, cleanup, reproducibility and interview questions

Objective and production-minded architecture

- Predict whether a synthetic shipment will be delayed using eight leakage-reviewed features.
- Train two simple models: logistic regression baseline and random forest.
- Use local Python 3.12 for low-cost training, while Azure ML is used for tracking, data/model assets and optional batch serving.
- Log parameters, metrics, model, feature list, dataset fingerprint and code version through MLflow.
- Register the selected model, produce risk_score, then prove one batch scoring path works.
- Delete temporary endpoint and compute immediately after validation.

```
Local CSV -> local training -> MLflow tracking in Azure ML
                        -> compare candidates -> register selected model
                        -> local risk_score
                        -> optional Azure batch endpoint -> predictions
```

Cost guardrail: the final batch compute is scale-to-zero (min_instances=0, max_instances=1). Do not leave temporary serving resources running after the demo.

Data contract and leakage review

Training features are intentionally simple and explainable. The label is is_delayed. None of the input features should contain information only known after the shipment has already been delayed.

origin_region	**destination_region**	**carrier**	**distance_km**
order_hour	**weekday**	**priority**	**historical_delay_rate**

Leakage rule: do not add an outcome-derived field such as actual_arrival_delay_minutes, final_delivery_status, or a feature calculated after the prediction decision point.

Project structure - core files

```
POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT/
  pyproject.toml             Poetry dependencies
  poetry.toml               .venv inside project
  .env                      Azure workspace configuration
  ml/
    azure_utils.py          auth + MLClient helper
    generate_data.py        synthetic train/scoring CSV
    verify_config.py        Azure workspace + MLflow URI check
    register_data.py        Azure ML data asset
    train.py                LR + random forest + MLflow
    compare_runs.py         compare candidates
    register_model.py       register selected model
    score.py                produce risk_score
    verify_poc.py           final validation
  tests/                    deterministic data + feature contract
```

There is no smoke_test.py in the final POC-07 repository. Its role is covered by verify_config.py, verify_poc.py and pytest. This guide uses the actual final file names rather than inventing a script.

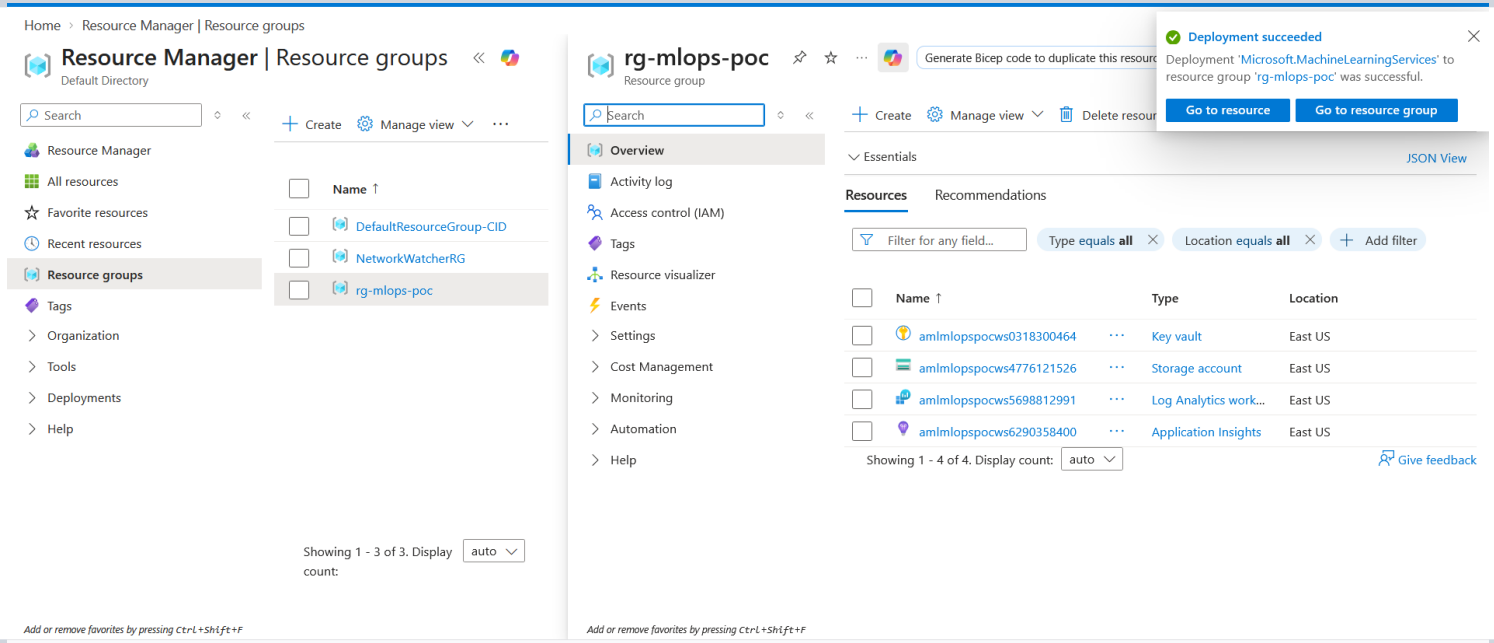
Project structure - Azure batch serving files

```
azure/batch/
  deploy_batch.py
  invoke_batch.py
  cleanup_batch.py
  environment/conda.yaml      explicit Python 3.11 serving env
  code/batch_driver.py        explicit init()/run() batch scorer

data/train/shipments.csv
data/scoring/shipments_scoring.csv
outputs/
docs/
.github/workflows/ci.yml
```

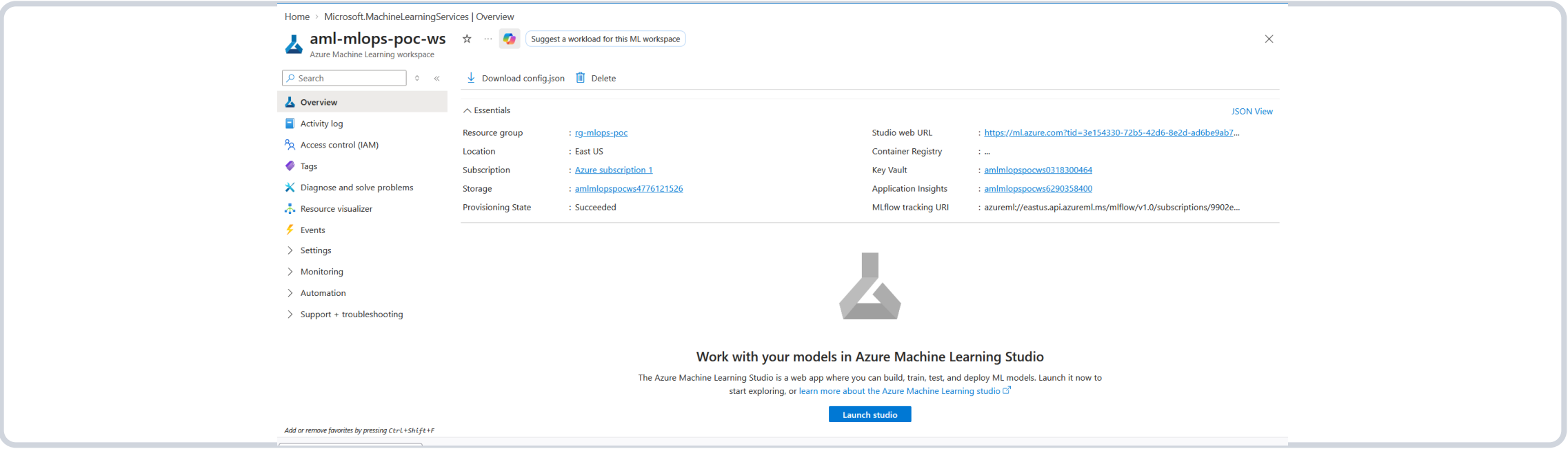
- deploy_batch.py creates/reuses scale-to-zero compute, endpoint, custom environment and fresh deployment.
- invoke_batch.py prepares an exact eight-feature CSV, invokes the deployment and downloads results.
- cleanup_batch.py removes endpoint and compute after validation.
- The remote scoring environment is intentionally separate from local Poetry/Python 3.12.

Create the Azure resource group



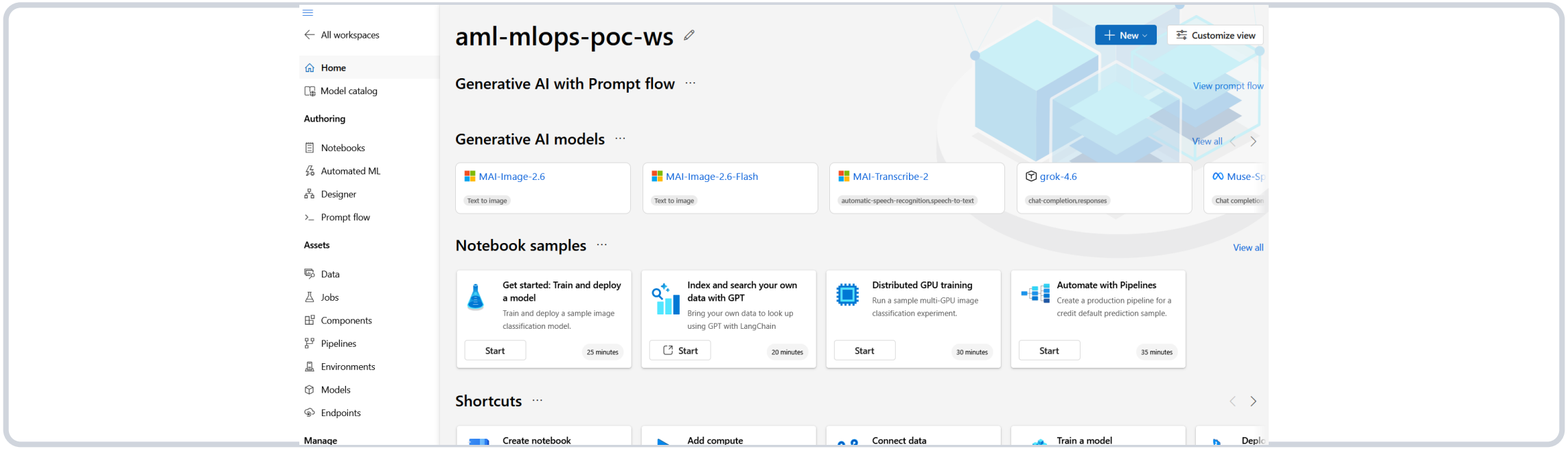
- Azure Portal -> Resource groups -> Create.
- Use one POC resource group so all resources can be deleted together later.
- Actual completed POC used resource group rg-mlops-poc.

Create the Azure Machine Learning workspace



- Azure Portal -> Create a resource -> Machine Learning -> Azure Machine Learning workspace.
- Choose the POC resource group and a supported region. The completed workspace name is aml-mlops-poc-ws.
- Wait until provisioning state is Succeeded.

Verify Azure ML workspace details



- Open the workspace and confirm subscription, resource group, region and provisioning state.
- Launch Azure ML Studio. The workspace provides access to Data, Jobs/Experiments, Models and Endpoints.

Create the Poetry environment

```
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run python --version
Python 3.12.11

C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run python -c "import mlflow, sklearn, pandas; p
rint('Poetry environment OK')"
Poetry environment OK

C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run python -m ml.generate_data
[OK] Training data: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\train\shipments.csv (2000 rows
)
[OK] Scoring data:  C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\scoring\shipments_scoring.csv
(40 rows)
[INFO] Delay rate: 0.349

C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>call .venv\Scripts\activate.bat

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Run the POC with Poetry instead of manually creating a pip-only venv.
- poetry.toml keeps the virtual environment inside the project at .venv.

```
poetry --version
py -3.12 --version
poetry env use 3.12
poetry install
poetry env info
poetry run python --version
```

Generate synthetic shipment data

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.generate_data
[OK] Training data: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\train\shipments.csv (2000 rows
)
[OK] Scoring data:  C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\scoring\shipments_scoring.csv
(40 rows)
[INFO] Delay rate: 0.349

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Creates deterministic training data and a small scoring dataset.
- Expected outputs: data\train\shipments.csv and data\scoring\shipments_scoring.csv.

```
poetry run python -m ml.generate_data
```

Train two models with local MLflow tracking

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.train --tracking local
2026/09/06 15:37:32 INFO mlflow.tracking.fluent: Experiment with name 'poc07-shipment-delay' does not exist. Creating a new experiment.
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\mlflow\types\utils.py:452: User
Warning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing va
lues at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model sch
ema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) wh
enever these columns may have missing values. See `Handling Integers With Missing Values <https://www.mlflow.org/docs/latest/models.html#handling-integers-w
ith-missing-values>`_ for more details.
  warnings.warn(
Downloading artifacts: 100%|                                                                                                                          | 7/7 [00:00<00:00, 6903.39it/s]
[OK] logistic_regression: run=e6be388928964bd69c6c484417f3d8fa roc_auc=0.6471 f1=0.2456
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\mlflow\types\utils.py:452: User
Warning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing va
lues at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model sch
ema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) wh
enever these columns may have missing values. See `Handling Integers With Missing Values <https://www.mlflow.org/docs/latest/models.html#handling-integers-w
ith-missing-values>`_ for more details.
  warnings.warn(
Downloading artifacts: 100%|                                                                                                                          | 7/7 [00:00<00:00, 2857.71it/s]
[OK] random_forest: run=532af7ecc057417bbae74bc6db052bc3 roc_auc=0.6542 f1=0.2584
[SELECTED] random_forest with ROC AUC 0.6542
[OK] Selection manifest: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\outputs\selected_model.json

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Trains logistic regression and random forest, logs parameters/metrics/model metadata, and selects the best validation candidate.
- Run local tracking first so data/model code problems are separated from Azure connectivity problems.

```
poetry run python -m ml.train --tracking local
```

Compare candidate models

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pyth
n -m ml.compare_runs
MODEL COMPARISON
================================================================================
model                     roc_auc         f1  precision     recall             run_id
logistic_regression        0.6471     0.2456     0.6774     0.1500 e6be388928964bd69c
random_forest              0.6542     0.2584     0.6053     0.1643 532af7ecc057417bba
================================================================================
Selected: random_forest (highest validation roc_auc; f1 used as tie-breaker)

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Compare logged metrics and verify both candidate runs appear.
- Selection is based on validation ROC AUC with F1 used as tie-breaker.

```
poetry run python -m ml.compare_runs
```

Score locally and create risk_score

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.score --tracking local
[OK] Wrote 40 predictions: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\outputs\predictions.csv
[INFO] Mean risk score=0.3580; mean latency=0.608 ms/row

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Runs the selected local model against scoring rows.
- Outputs predictions.csv and scoring_metrics.json. risk_score is the estimated probability of delayed class 1.

```
poetry run python -m ml.score --tracking local
```

Run local validation and tests

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.verify_poc --tracking local
[PASS] Synthetic training data exists
[PASS] All required features and label exist
[PASS] Model selection manifest exists
[PASS] At least two MLflow runs were logged
[PASS] A selected run is recorded
[PASS] Feature list matches leakage-reviewed design
[PASS] A scoring path produced predictions
[PASS] risk_score is present
[PASS] risk_score is within [0, 1]
[PASS] Leakage review is documented

RESULT: PASS

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytes
t -q
..                                                                                                                   [100%]
2 passed in 0.40s

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Final local gate: verify required artifacts/runs and run automated tests.
- This is the POC-07 smoke-test equivalent.

```
poetry run python -m ml.verify_poc --tracking local
poetry run pytest -q
```

Generate data again before Azure flow

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.generate_data
[OK] Training data: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\train\shipments.csv (2000 rows
)
[OK] Scoring data:  C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\scoring\shipments_scoring.csv
(40 rows)
[INFO] Delay rate: 0.349

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Re-run deterministic generation before publishing data to Azure so the dataset used in the cloud flow is known and reproducible.

```
poetry run python -m ml.generate_data
```

Configure .env for the Azure workspace

```
AZURE_SUBSCRIPTION_ID=<your-subscription-id>
AZURE_RESOURCE_GROUP=rg-mlops-poc
AZUREML_WORKSPACE_NAME=aml-mlops-poc-ws
AZURE_ML_WORKSPACE_NAME=aml-mlops-poc-ws
AZURE_AUTH_MODE=cli

MLFLOW_EXPERIMENT_NAME=poc07-shipment-delay
AZUREML_DATA_ASSET_NAME=shipment-delay-synthetic
AZUREML_MODEL_NAME=shipment-delay-model
AZUREML_BATCH_COMPUTE_NAME=cpu-poc07
AZUREML_BATCH_ENDPOINT_NAME=<unique-endpoint-name>
AZUREML_BATCH_DEPLOYMENT_NAME=blue-v2
AZUREML_BATCH_ENVIRONMENT_NAME=poc07-batch-mlflow-py311
```

Keep secrets out of Git. The local beginner flow uses Azure CLI authentication, so no client secret is required.

Error faced #1 - verify_config.py failed after browser authentication

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.verify_config
[OK] Required .env settings are present.
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\msal\oauth2cli\oauth2.py:461: U
serWarning: response_mode='form_post' is recommended for better security. See https://www.rfc-editor.org/rfc/rfc9700.html#section-4.3.1
  warnings.warn(
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\ml\verify_config.py", line 36, in <module>
    main()
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\ml\verify_config.py", line 27, in main
    ws = client.workspaces.get(client.workspace_name)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\_telemetry\
activity.py", line 288, in wrapper
    return f(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\core\tracing\deco
rator.py", line 119, in wrapper_use_tracer
    return func(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\operations\
_workspace_operations.py", line 143, in get
    return super().get(workspace_name=name, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\operations\
_workspace_operations_base.py", line 91, in get
    obj = self._operation.get(resource_group, workspace_name)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\core\tracing\deco
rator.py", line 119, in wrapper_use_tracer
    return func(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\_restclient
\v2024_10_01_preview_tsp\operations\_operations.py", line 10422, in get
    map_error(status_code=response.status_code, response=response, error_map=error_map)
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\core\exceptions.p
y", line 163, in map_error
    raise error
```

- Symptom: browser authentication completed, but workspace lookup still failed in Python.
- The workspace itself was valid; Azure CLI could find it. That isolated the issue to local credential selection/environment loading.
- Root cause in the original helper: AZURE_AUTH_MODE=cli did not actually select AzureCliCredential, and old environment variables could override the corrected .env.

Fix #1 - make CLI authentication explicit and reload .env correctly

```
# ml/azure_utils.py - corrected behavior
load_dotenv(dotenv_path=ENV_FILE, override=True)

if mode == "cli":
    return AzureCliCredential()
elif mode == "interactive":
    return InteractiveBrowserCredential(...)
else:
    return DefaultAzureCredential(...)
```

```
az login
az account set --subscription "Azure subscription 1"
az account show -o table
poetry run python -c "import ml.azure_utils as a; print(a.__file__)"
poetry run python -m ml.verify_config
```

Critical verification: verify_config.py should report Credential type: AzureCliCredential and Auth mode: cli. If a localhost browser still opens, you are running old code or an old imported module.

verify_config.py succeeds after authentication fix

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -c "import ml.azure_utils as a; print(a.__file__)"
C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\ml\azure_utils.py

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.verify_config
[OK] Required .env settings are present.

Effective configuration used by Python:
  .env file       : C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.env
  Subscription ID : 9902e04b...672c
  Resource group  : rg-mlops-poc
  Workspace       : aml-mlops-poc-ws
  Tenant ID       : <Azure CLI/default>
  Auth mode       : cli

[1/4] Building Azure credential...
[OK] Credential type: AzureCliCredential

[2/4] Testing Azure Resource Manager authentication...
[OK] Azure authentication succeeded.

[3/4] Looking up the configured Azure ML workspace...
[OK] Connected to Azure ML workspace: aml-mlops-poc-ws
[OK] Location: eastus
[OK] Resource group: rg-mlops-poc
[OK] Provisioning state: <not returned>

[4/4] Discovering Azure ML MLflow tracking URI...
[OK] MLflow tracking URI: azureml://eastus.api.azureml.ms/mlflow/v2.0/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-mlops-poc/provide
rs/Microsoft.MachineLearningServices/workspaces/aml-mlops-poc-ws

[SUCCESS] Azure ML workspace + MLflow configuration verified.
Next: poetry run python -m ml.register_data

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Python now reads the intended subscription, resource group and workspace values.
- Workspace lookup succeeds and the Azure ML MLflow tracking URI can be discovered.
- This is the gate before registering data or logging Azure MLflow experiments.

Register training data as an Azure ML data asset

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.generate_data
[OK] Training data: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\train\shipments.csv (2000 rows
)
[OK] Scoring data:  C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\data\scoring\shipments_scoring.csv
(40 rows)
[INFO] Delay rate: 0.349

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Registers/version-controls the synthetic training CSV in Azure ML workspace-managed storage.
- Use the data asset as the reproducible dataset reference recorded with the model.

```
poetry run python -m ml.register_data
```

Confirm the registered data asset

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.register_data
Uploading shipments.csv (< 1 MB): 100%|##########################################################################################| 125k/125k [00:00<00:00, 158kB/s]


[OK] Registered data asset: shipment-delay-synthetic:4a779922364d
[INFO] Azure ML uploaded the local file to workspace-managed Azure Storage.

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Open Azure ML Studio -> Data and verify the asset name/version.
- Registration confirms the local dataset has a cloud-visible lineage reference.

Track training runs in Azure ML through MLflow

- Logs two Azure MLflow runs using the workspace tracking URI.
- Each run records parameters, validation metrics, model artifact, feature list, dataset fingerprint and code version.

```
poetry run python -m ml.train --tracking azure
```

Verify experiment runs in Azure ML Studio

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.compare_runs
MODEL COMPARISON
================================================================================================
model                       roc_auc         f1  precision     recall             run_id
logistic_regression          0.6471     0.2456     0.6774     0.1500 e4b89f55-c3c2-446b
random_forest                0.6542     0.2584     0.6053     0.1643 e7eebed1-712e-4219
================================================================================================
Selected: random_forest (highest validation roc_auc; f1 used as tie-breaker)

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Open Jobs/Experiments and compare logistic regression vs random forest.
- Use the same selection rule as local validation.

```
poetry run python -m ml.compare_runs
```

Register the selected model

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.register_model --tracking azure
Successfully registered model 'shipment-delay-model'.
2026/09/06 17:30:13 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model version to finish creation. Model name: shipment-de
lay-model, version 1
Created version '1' of model 'shipment-delay-model'.
[OK] Registered model: shipment-delay-model version 1
[OK] Registration manifest: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\outputs\registered_model.js
on

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Registers the chosen MLflow model into Azure ML model registry.
- Model metadata should retain metric, dataset reference, code version and limitations.

```
poetry run python -m ml.register_model --tracking azure
```

Verify the Azure ML registered model version

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.score --tracking azure
Downloading artifacts: 100%|                                                                                                                                | 7/7 [00:03<00:00,  1.89it/s]
[OK] Wrote 40 predictions: C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\outputs\predictions.csv
[INFO] Mean risk score=0.3580; mean latency=0.474 ms/row

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Azure ML Studio -> Models -> shipment-delay-model.
- Confirm the registered version exists before attempting batch deployment.

Generate Azure-tracked risk scores

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n -m ml.verify_poc --tracking azure --check-registry
[PASS] Synthetic training data exists
[PASS] All required features and label exist
[PASS] Model selection manifest exists
[PASS] At least two MLflow runs were logged
[PASS] A selected run is recorded
[PASS] Feature list matches leakage-reviewed design
[PASS] A scoring path produced predictions
[PASS] risk_score is present
[PASS] risk_score is within [0, 1]
[PASS] Registration manifest exists
[PASS] Registered model version is queryable
[PASS] Leakage review is documented

RESULT: PASS

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- Produces risk_score and validates that the selected Azure-tracked model/registry artifacts are queryable.
- An AI agent may explain the prediction but must not claim causal certainty.

```
poetry run python -m ml.score --tracking azure
poetry run python -m ml.verify_poc --tracking azure --check-registry
```

Initial batch deployment succeeds, but invocation later fails

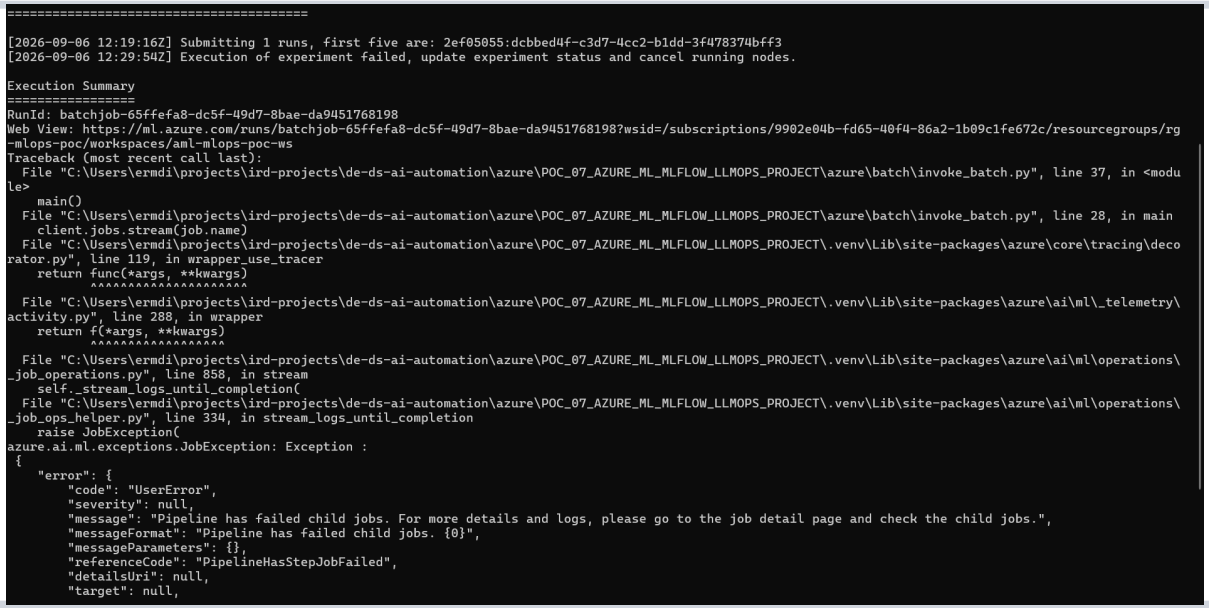
```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n azure\batch\deploy_batch.py
[1/3] Creating/reusing scale-to-zero compute cpu-poc07 (Standard_DS2_v2)
[2/3] Creating/reusing batch endpoint shipment-delay-batch-1fe672c
[3/3] Deploying latest registered MLflow model shipment-delay-model
[OK] Batch deployment ready. Endpoint=shipment-delay-batch-1fe672c, deployment=blue

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

- deploy_batch.py created the compute/endpoint/deployment resource successfully.
- Important lesson: successful deployment resource creation does not prove the serving image can build or that the scoring child job will run.
- The first implementation relied too heavily on Azure inferring an MLflow serving environment.

```
poetry run python azure\batch\deploy_batch.py
```

Error faced #2 - invoke_batch.py parent pipeline fails



```
=======================================

[2026-09-06 12:19:16Z] Submitting 1 runs, first five are: 2ef05055:dcbbed4f-c3d7-4cc2-b1dd-3f478374bff3
[2026-09-06 12:29:54Z] Execution of experiment failed, update experiment status and cancel running nodes.

Execution Summary
=================
RunId: batchjob-65ffefa8-dc5f-49d7-8bae-da9451768198
Web View: https://ml.azure.com/runs/batchjob-65ffefa8-dc5f-49d7-8bae-da9451768198?wsid=/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourcegroups/rg
-mlops-poc/workspaces/aml-mlops-poc-ws
Traceback (most recent call last):
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\azure\batch\invoke_batch.py", line 37, in <modu
le>
    main()
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\azure\batch\invoke_batch.py", line 28, in main
    client.jobs.stream(job.name)
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\core\tracing\deco
rator.py", line 119, in wrapper_use_tracer
    return func(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^

  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\_telemetry\
activity.py", line 288, in wrapper
    return f(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^

  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\operations\
_job_operations.py", line 858, in stream
    self._stream_logs_until_completion(
  File "C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT\.venv\Lib\site-packages\azure\ai\ml\operations\
_job_ops_helper.py", line 334, in stream_logs_until_completion
    raise JobException(
azure.ai.ml.exceptions.JobException: Exception :
 {
    "error": {
        "code": "UserError",
        "severity": null,
        "message": "Pipeline has failed child jobs. For more details and logs, please go to the job detail page and check the child jobs.",
        "messageFormat": "Pipeline has failed child jobs. {0}",
        "messageParameters": {},
        "referenceCode": "PipelineHasStepJobFailed",
        "detailsUri": null,
        "target": null,
```

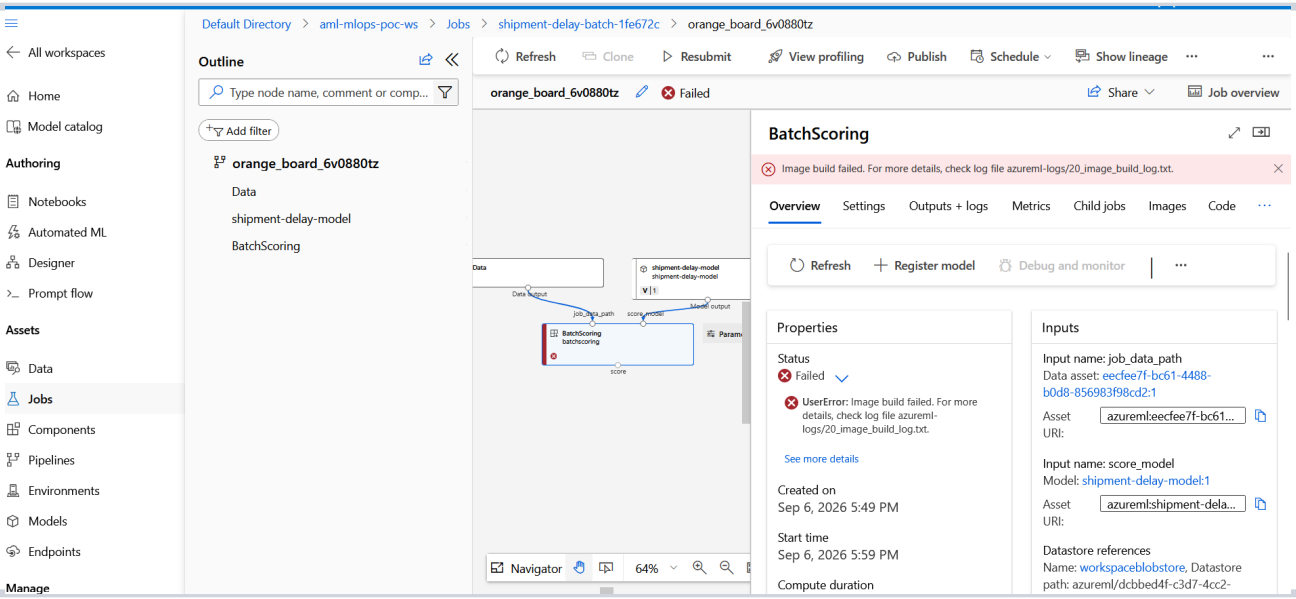
- Symptom: invoke_batch.py uploads input and submits a batch job, then client.jobs.stream() raises JobException.
- The parent error PipelineHasStepJobFailed is only a wrapper. The real failure must be diagnosed in the BatchScoring child job logs.
- Changing CSV columns alone cannot fix a failure that occurs before the scoring container starts.

Inspect the failed parent/child job instead of guessing


```
az ml job list ^
  --parent-job-name <PARENT_JOB_NAME> ^
  --resource-group rg-mlops-poc ^
  --workspace-name aml-mlops-poc-ws ^
  --query "[].{Name:name,DisplayName:display_name,Status:status}" -o table

az ml job show --name <PARENT_JOB_NAME> ^
  --resource-group rg-mlops-poc ^
  --workspace-name aml-mlops-poc-ws --web
```

Root cause revealed - BatchScoring image build failed



- Azure ML Studio explicitly reports: Image build failed and points to azureml-logs/20_image_build_log.txt.
- This proves the model never reached CSV scoring. The environment/container image failed first.
- The failed job metadata still referenced the old deployment blue, timeout 300 and INFO logging - evidence that the invocation was still using the old serving deployment.

Fix #2 - use a fresh explicit batch serving environment

```
.env
AZUREML_BATCH_DEPLOYMENT_NAME=blue-v2
AZUREML_BATCH_ENVIRONMENT_NAME=poc07-batch-mlflow-py311

azure/batch/
  deploy_batch.py
  environment/conda.yaml
  code/batch_driver.py
  invoke_batch.py
```

- Keep local development on Poetry/Python 3.12, but use an explicit Python 3.11 Azure batch environment.
- Pin the remote runtime dependencies and include azureml-core required by the batch scoring runtime.
- Provide a custom batch_driver.py with init() and run(mini_batch) so Azure does not have to infer how to serve the logged model.
- Use a new deployment name blue-v2 to guarantee the endpoint invokes the corrected configuration.

Remote conda.yaml used by the corrected deployment

```
name: poc07-batch-mlflow-py311
channels:
  - conda-forge
dependencies:
  - python=3.11
  - pip<=24.3
  - pip:
      - mlflow==2.22.5
      - scikit-learn==1.7.2
      - pandas>=2.2,<3
      - numpy>=1.26,<3
      - azureml-core==1.61.0.post4
```

The serving environment is separate from the local Poetry environment. This is deliberate and avoids automatic image-generation incompatibilities.

batch_driver.py - explicit scoring contract

```
def init():
    # find MLmodel under AZUREML_MODEL_DIR
    # load registered model with mlflow.pyfunc.load_model(...)

def run(mini_batch):
    # read each CSV
    # select exactly the 8 FEATURES
    # model.predict(model_input)
    # return DataFrame with risk_score and is_delayed_pred
```

- The driver protects the model signature by selecting only the eight expected input features.
- It turns the model output into stable batch rows with risk_score and prediction class.
- This explicit scorer makes failures easier to diagnose than automatic MLflow serving inference.

invoke_batch.py - final safe input preparation

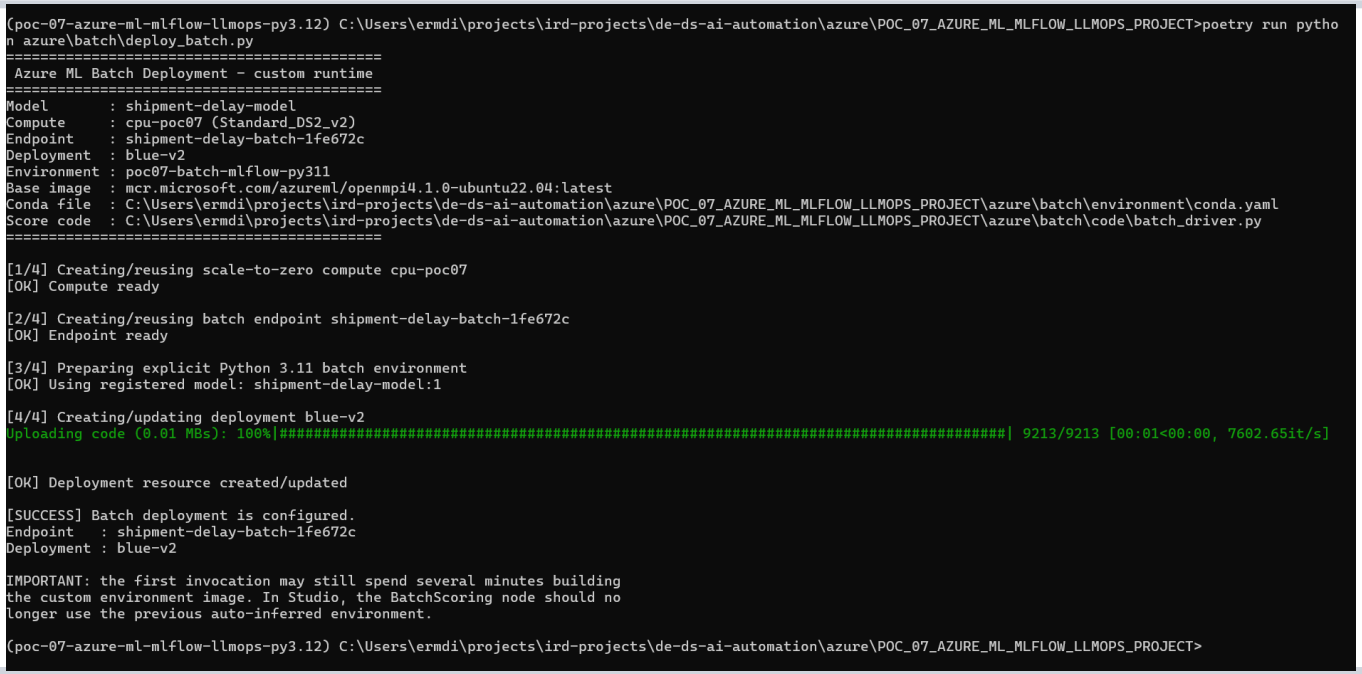
```
source = SCORING_DIR / "shipments_scoring.csv"
df = pd.read_csv(source)
batch_df = df[FEATURES].copy()

batch_dir = OUTPUT_DIR / "batch_input"
batch_df.to_csv(batch_dir / "shipments_batch.csv", index=False)

job = client.batch_endpoints.invoke(
    endpoint_name=endpoint_name,
    deployment_name=deployment_name,
    input=Input(type=AssetTypes.URI_FOLDER, path=batch_dir.resolve()),
)
```

The batch input folder is cleaned first so stale CSVs cannot be uploaded accidentally. shipment_id and is_delayed are excluded from the model feature matrix.

Corrected blue-v2 deployment succeeds




```
poetry run python azure\batch\deploy_batch.py
```

- The deployment now creates an explicit environment and scoring script instead of relying on automatic image inference.
- The fresh deployment name prevents accidental reuse of the broken old blue deployment.

Verify the corrected deployment resource

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>az ml batch-depl
oyment show ^
More?   --name blue-v2 ^
More?   --endpoint-name shipment-delay-batch-1fe672c ^
More?   --resource-group rg-mlops-poc ^
More?   --workspace-name aml-mlops-poc-ws ^
More?   --query "{Name:name,ProvisioningState:provisioning_state,Compute:compute}" ^
More?   -o table
Name     ProvisioningState    Compute
-------  -------------------  ------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------
blue-v2  Succeeded            azureml:/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-mlops-poc/providers/Microsoft.MachineLearningSer
vices/workspaces/aml-mlops-poc-ws/computes/cpu-poc07

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

```
az ml batch-deployment show ^
  --name blue-v2 ^
  --endpoint-name <your-endpoint-name> ^
  --resource-group rg-mlops-poc ^
  --workspace-name aml-mlops-poc-ws -o table
```

Verify endpoint default deployment points to blue-v2

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>az ml batch-depl
oyment show ^
More?   --name blue-v2 ^
More?   --endpoint-name shipment-delay-batch-1fe672c ^
More?   --resource-group rg-mlops-poc ^
More?   --workspace-name aml-mlops-poc-ws ^
More?   --query "{Name:name,ProvisioningState:provisioning_state,Compute:compute}" ^
More?   -o table
Name     ProvisioningState    Compute
-------  -------------------  ------------------------------------------------------------------------------------------------------------------------
------------------------------------------------------
blue-v2  Succeeded            azureml:/subscriptions/9902e04b-fd65-40f4-86a2-1b09c1fe672c/resourceGroups/rg-mlops-poc/providers/Microsoft.MachineLearningSer
vices/workspaces/aml-mlops-poc-ws/computes/cpu-poc07

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>az ml batch-endp
oint show ^
More?   --name shipment-delay-batch-1fe672c ^
More?   --resource-group rg-mlops-poc ^
More?   --workspace-name aml-mlops-poc-ws ^
More?   --query "{Endpoint:name,DefaultDeployment:defaults.deployment_name}" ^
More?   -o table
Endpoint                     DefaultDeployment
---------------------------  -------------------
shipment-delay-batch-1fe672c  blue-v2
```

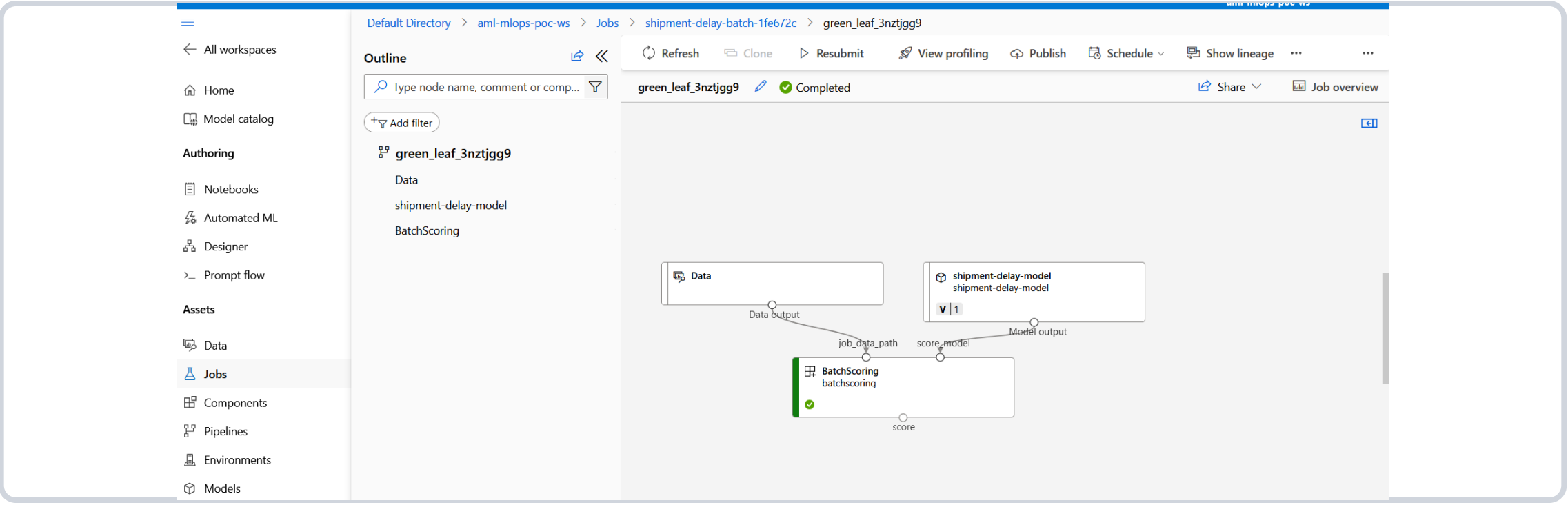
```
az ml batch-endpoint show ^
  --name <your-endpoint-name> ^
  --resource-group rg-mlops-poc ^
  --workspace-name aml-mlops-poc-ws ^
  --query "{Endpoint:name,DefaultDeployment:defaults.deployment_name}" -o table
```

Final invoke succeeds


```
poetry run python azure\batch\invoke_batch.py
```

- Input is uploaded, the Azure child scoring job runs successfully, and predictions are downloaded.
- This satisfies the source POC requirement that one cloud scoring path works.

Azure ML Studio shows the completed batch scoring job



- The BatchScoring node is green/completed rather than failed.
- This is the final visual proof that the corrected serving image, model, input and scoring code work together.

Cleanup immediately after validation

```
(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>poetry run pytho
n azure\batch\cleanup_batch.py
[DELETE] Batch endpoint: shipment-delay-batch-1fe672c
.........[DELETE] Compute: cpu-poc07
[OK] Batch serving resources deleted. Registered model/data/workspace remain.

(poc-07-azure-ml-mlflow-llmops-py3.12) C:\Users\ermdi\projects\ird-projects\de-ds-ai-automation\azure\POC_07_AZURE_ML_MLFLOW_LLMOPS_PROJECT>
```

```
poetry run python azure\batch\cleanup_batch.py
```

- Delete the temporary batch endpoint and AML compute after collecting evidence/screenshots.
- For a full teardown, delete model/data assets you do not need, then delete the entire POC resource group.
- Keep only the source repo, screenshots, result evidence and documentation needed for future reference.

Complete command sequence - local to Azure

```
poetry env use 3.12
poetry install
poetry run python -m ml.generate_data
poetry run python -m ml.train --tracking local
poetry run python -m ml.compare_runs
poetry run python -m ml.score --tracking local
poetry run python -m ml.verify_poc --tracking local
poetry run pytest -q

az login
az account set --subscription "Azure subscription 1"
poetry run python -m ml.verify_config
poetry run python -m ml.register_data
poetry run python -m ml.train --tracking azure
poetry run python -m ml.compare_runs
poetry run python -m ml.register_model --tracking azure
poetry run python -m ml.score --tracking azure
poetry run python -m ml.verify_poc --tracking azure --check-registry
```

Complete command sequence - batch serving

```
# .env must point to fresh corrected deployment
AZUREML_BATCH_DEPLOYMENT_NAME=blue-v2
AZUREML_BATCH_ENVIRONMENT_NAME=poc07-batch-mlflow-py311

poetry run python azure\batch\deploy_batch.py

az ml batch-deployment show ^
  --name blue-v2 ^
  --endpoint-name <your-endpoint-name> ^
  --resource-group rg-mlops-poc ^
  --workspace-name aml-mlops-poc-ws -o table

poetry run python azure\batch\invoke_batch.py
poetry run python azure\batch\cleanup_batch.py
```

Never continue invoking a known-broken deployment. Verify the deployment name in .env and in the Azure job metadata before debugging anything else.

Troubleshooting matrix - problems encountered in this POC

verify_config opens browser then fails	Credential path mismatch	Set AZURE_AUTH_MODE=cli; use AzureCliCredential
.env edits appear ignored	Existing process env overrides file	load_dotenv(..., override=True)
Workspace not found in Python	Wrong effective subscription/RG/workspace	Print effective values; verify with az ml workspace show
Parent batch job fails	Child BatchScoring failed	Inspect child job; parent exception is not root cause
Image build failed	Serving environment could not build	Use explicit Python 3.11 conda environment + custom sc
New invoke still uses old settings	Old deployment blue is still selected	Use fresh blue-v2 and verify endpoint default
Batch input schema mismatch	Extra/missing model columns	Create clean folder with exactly 8 FEATURES
Compute/quota issue	VM cannot allocate	Choose an allowed small CPU SKU, keep scale-to-zero

Monitoring and reproducibility checklist

- Scoring latency: batch job duration, startup time and mini-batch processing time.
- Errors: failed child jobs, model load errors, schema failures, storage access errors.
- Prediction distribution: monitor shifts in average risk_score and delayed-class rate.
- Feature drift: compare recent origin/destination/carrier/distance/hour/weekday/priority/history distributions with training data.
- Data freshness: prove that the scoring dataset is recent enough for the business decision.
- Reproducibility: a clean clone should be able to poetry install, generate data, train, track, register and score with the same class of results.
- CI concept: run deterministic generation + tests + local training checks before allowing cloud deployment changes.

Interview revision - key questions from POC-07

Q1. MLflow tracking vs model registry?

Tracking stores runs, parameters, metrics and artifacts. Registry manages promoted/versioned model assets and lifecycle metadata.

Q2. Batch vs online inference?

Batch scores many records asynchronously and is cost-effective for scheduled/data-engineering workloads. Online serves low-latency request/response traffic.

Q3. What is data leakage?

Using information unavailable at prediction time, which makes validation unrealistically optimistic.

Q4. How do you version data and models?

Use Azure ML data/model asset versions plus dataset fingerprints, run IDs, metrics and code version metadata.

Q5. Model monitoring vs pipeline monitoring?

Model monitoring watches prediction quality/distribution/drift; pipeline monitoring watches orchestration, data movement, failures and freshness.

Q6. How does a Data Engineer support ML reproducibility?

Version data contracts, automate deterministic pipelines, capture lineage, enforce schemas, and provide repeatable environments/CI.

POC completion checklist

- ☑ Resource group and Azure ML workspace created and verified.
- ☑ Poetry environment installs successfully with Python 3.12 locally.
- ☑ Synthetic train/scoring datasets generated; feature leakage review documented.
- ☑ Two candidate models trained and compared.
- ☑ Azure MLflow experiment runs visible in workspace.
- ☑ Selected model registered with model version/metadata.
- ☑ risk_score generated successfully.
- ☑ Batch endpoint/deployment created using explicit custom serving environment.
- ☑ Batch invocation completed successfully in Azure ML Studio.
- ☑ Endpoint and compute deleted after the demo.
- ☑ Final repository + screenshots + this reference guide retained for repeatability.

Final beginner takeaway

- Build locally first, then add Azure tracking. This makes cloud errors much easier to isolate.
- Always print the effective configuration Python is actually using; do not assume editing .env changed the current process.
- When a batch parent job fails, investigate the failed child job and the first real error in its logs.
- A deployment resource can succeed while the serving image later fails to build. Treat image build, compute allocation, model load and scoring as separate stages.
- Use fresh deployment names during troubleshooting so you can prove Azure is executing the corrected configuration.
- Capture both failure and success screenshots. They are valuable evidence of how the POC was debugged and made production-minded.
- Clean up temporary compute/endpoints immediately after the final validation.

Successful end state: Azure ML tracks two experiments, the selected model is registered, risk_score is produced, the corrected blue-v2 batch job completes, and temporary resources are cleaned up.